

Natural Language Processing

↳ Spoken by human being.

→ Natural language is not generated by a system

→ Human language
{

 written
 spoken

filler words & pet phrases (e.g. hmm, uh, haan)

↓
 {disfluency} → Some times ungrammatical.

NLP Toolkit → Spacy, NLTK, Stanza.

Kinds of NLP tasks:

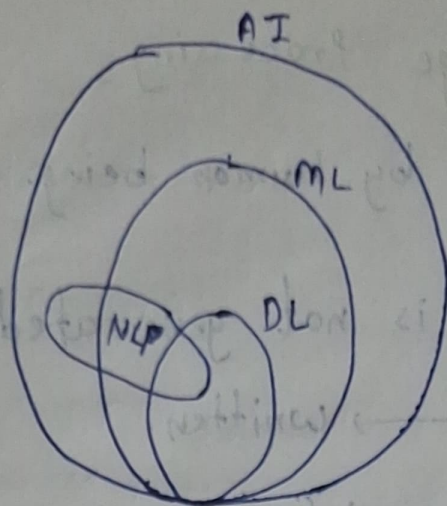
↳ statistical language model (next token prediction)

↳ Text classification using (ML/DL techniques)

↳ sequence labeling / Sequence Tagging / Token classification

↳ sequence generation.

Implement all these learning in a project



ML $\rightarrow$ feature

DL $\rightarrow$

ML $\rightarrow$ Train
L, Test

inference $\rightarrow$ Prediction.

Inductive bias $\rightarrow$ Assumptions made by Model

Tokenizer $\rightarrow$ convert the text to processable
(pretrained) & units

() it return token-ids.

Computational Linguistic $\rightarrow$ Analysis of language,
"CL"

LLMs $\rightarrow$ Written text
 $\rightarrow$ spoken text / speech
 $\rightarrow$ visual (image/video)

ASR output / transcript $\rightarrow$ text-based format.

$\Rightarrow$ NLP is a subfield of AI and CL which applies different techniques to NL

Turing Test $\rightarrow$ evaluator has to identify by the (transcript) whether on the other side is human or machine.

transcript $\rightarrow$ spoken text

$\Rightarrow$ 1970's 1st translation system

Georgetown - IBM translation system
(701) (200 sentences)

(Russian to English translation).

1) Bilingual dictionary (word to word translation)

2) Grammar rules about structure

Pronoun	noun	verb
mein	ghar	gaya.
I	went	home
Pronoun	verb	noun

• POS (Parts of speech) tags. (Rules)

• chunking / phrasing $\rightarrow$ (RAE)

Named Entity Recognition

Production Rule

$S \rightarrow NP \ VP$

$NP \rightarrow DET? JJ? NN | PRP$

$VP \rightarrow V \ NP$

($S \rightarrow$ sentence)

($NP \rightarrow$ noun phrase)

($VP \rightarrow$ verb phrase)

($NN \rightarrow$ noun)

AST $\Rightarrow$ Abstract Syntax tree.

$\Rightarrow S \rightarrow NP \ VP$

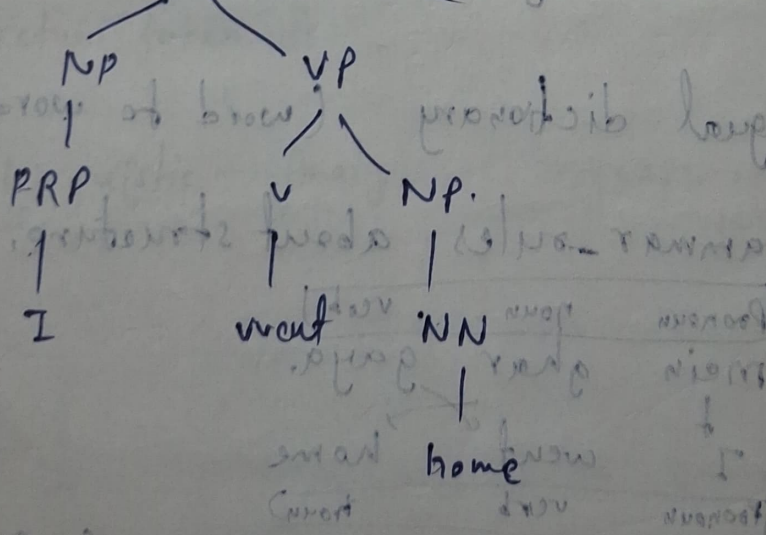
$NP \rightarrow DET? JJ? NN | PRP$

$VP \rightarrow V \ NP$

($DET \rightarrow$ Determiner)

($PRP \rightarrow$ Pronoun)

(constitency tree)



student ELIZA → first healthcare assistant

29/07/2020

NLP APPLICATIONS

- Q & A
- code Assistant
- [chatbot] virtual assistant
- Machine ~~Translation~~ Translation.
- Automatic speech recognition.
- Text to speech generation
- Summarization.
- speaker Identification
- Sentiment analysis.
- Text classification.

Transliteration → writing the letters or sounds of a word from one script into another script

eg. हिंदी → English (नमस्ते → Namaste)
 English → हिंदी (computer → कंप्यूटर)

Language Identification (LID)

input text $\rightarrow$ language.

$\Rightarrow$ Sequence labeling / sequence tagging / token classification.

Eng Eng Eng Eng Hindi Hindi Hindi Hindi
I am going to ghar. bahut dino baad

text / sequence classification.

span Identification

Noun Phrase Verb Phrase Preposition phrase

(The tall kid) (is going) (to the school)

B-NP I-NP I-NP B-VP I-VP B-PP I-PP I-PP

↑
constituent

sequence generation

I am going to ghari bahut dino baad.

$$3 \times 3 \times 3 \times \dots \times 3$$

$$= |\text{no. of tags/label}|^{\text{sequence length.}}$$

$$|S_E| = m$$

$$V_E = 10000$$

($V \rightarrow$ vocab vocabulary)

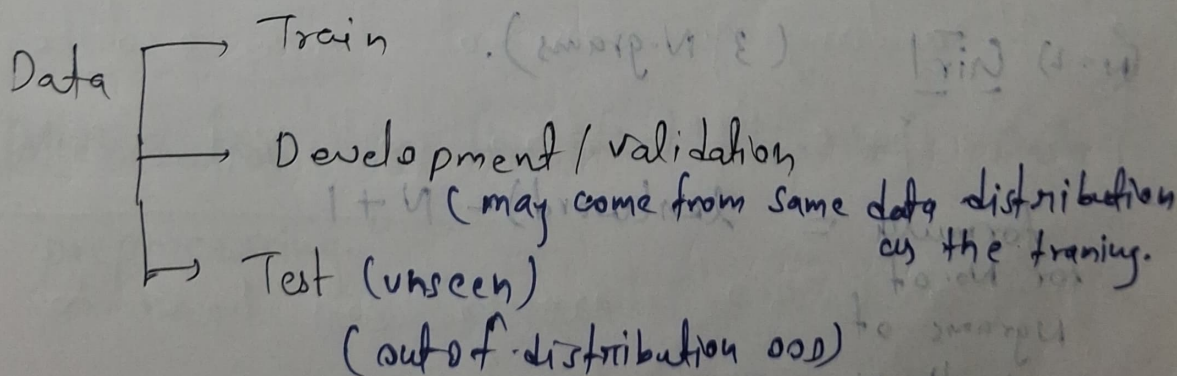
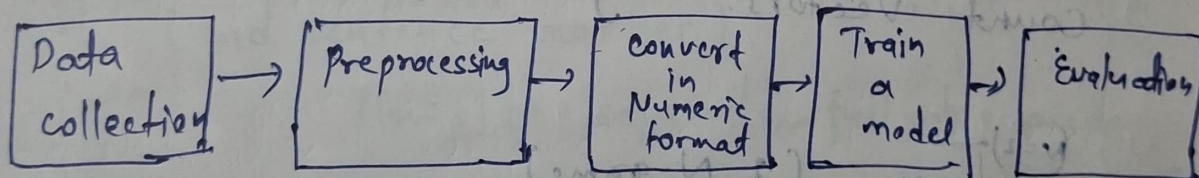
$$|S_H| = n$$

$$V_H = 100000$$

$$\text{no. of tags} = (V_E \times V_H)^n$$

|k - beam search|

NLP Pipeline



Tokenization

03/08/2026

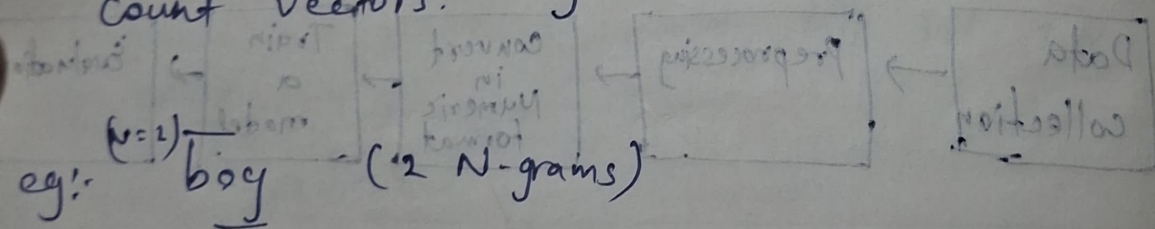
breaking a sentence into words.

breaking a longer string into smaller

Units

- character
 - word
 - sentence
 - Paragraph
 - Document
 - corpus/corpora
- fine grained.
- (granularity)
- coarse-grained
- collection of text/document
- corpora

TF-IDF vectors } for representation
count vectors.



$(N=2)$ Girl (3 N-grams) .

formula = $\text{len(word)} - N + 1$
for No. of
Ngrams of
length N

for sentence

eg

$$\text{Len(sentence)} = N + 1$$

eg: You are a student.

$$\text{Len(sentence)} = N + 1 \quad (N = 4)$$

$$(5) = 4 + 1 = 5$$

IPA (international phonetic alphabetic).

||
Syllable (units of speech, which can be spoken once at a time)

, (Pause, or listing elements)

; (combine clauses)

↳ each clause has a verb

• (end sentence marker)

How to separate full stop and

we use 'Regex'

Decimal point

"Tokenization is the first step of Data preprocessing."

⇒ We can simply tokenize a sentence using white space.

1) white space. ⇒ (space, it, is, a, carriage return)

read lines → list of words

read line → list of string.

1b → boundary.

1t → indentation (col, and row)

2) Tokenization By punctuation.

eg: <https://www.svnit.ac.in>.

3) Rule based / Regex based tokenization.

4) ML/DL Based tokenizer

★ If a language does not have a white space then how will you tokenize

given a series of character you have to find word boundaries

$c_1 c_2 c_3 \dots c_n$

we use a Dictionary a word (DP)

we can use "LCS" ^{longest} ~~least~~ common subsequence

[04/06/2026]

Regular Expression

why Regular Expression are required in ml

- matching pattern
- tokenization

wild card / special characters.

$+$ $\rightarrow$ one or more times

$*$ $\rightarrow$ zero or more times

$?$ $\rightarrow$ zero or one time

$\{m, n\}$ $\rightarrow$ m to n times. (m and n are inclusive)

$[abc]$ $\rightarrow$ set of characters.

a^+ $\rightarrow \{a, aa, aaa, \dots\}$

a^* $\rightarrow \{\epsilon, a, aa, aaa, \dots\}$

$a^{\{2, 4\}}$ $\rightarrow \{aa, aaaa, aaaaa\}$

$a\{2,2\} \rightarrow \{aa\}$

or

$a\{2\} \rightarrow \{aa\}$

$a? \rightarrow \{\epsilon, a\}$

$[abcd] \rightarrow \{a, b, c, d\}$

$[abcd]^+ \rightarrow \{a, b, c, d, aa, bb, cc, dd, ab, ac, ad, bc, bd, ed, \dots\}$

$\wedge \rightarrow$ start / negation of a set membership / character

$\$ \rightarrow$ end.

$(_)$ groups.

$[a-z]$ from a to z ($a, b, c, \dots, z$)

$[0-9]$ from 0 to 9 ($0, 1, 2, 3, \dots, 9$)

Unicode blocks of Devanagari

ASCII $\rightarrow$ have 256 character

$[a-zA-Z]$

$[a-zA-Z0-9_]$ (alphanumeric)

or

$\backslash w$

$\backslash d \rightarrow [0-9]$

$\backslash W \rightarrow [^ \backslash w]$ (non alphanumeric, $\rightarrow @, *, -, ', \dots$)

$^ [a-z] \rightarrow$ starts with (a to z)

$[^ a-z] \rightarrow$ does not contain (a to z)

$\backslash s \rightarrow$ white space

$\backslash S \rightarrow [^ \backslash s] \rightarrow$ non white space

$\backslash S+ \neq \backslash w+$

for matching special character

use escape $\backslash +$

for matching $+$ $\rightarrow \backslash +$
escape

for matching $*$ $\rightarrow \backslash *$

$^ \cdot + \backslash \cdot \$$ (finds the first string which ends with dot ($\cdot$))
(greedy operator)

eg:- I am at SVNIT & SURAT

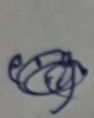
I have taken the boning NLP class:

$\rightarrow$ it will match both (greedy approach)

we have to use a non greedy approach

=) $\wedge \cdot * ? \wedge$
or
 $\wedge \cdot + ? \wedge$

Search & match functions



both returns index

match ~~rx~~ ~~add~~

eg:- $x = 'abcd'$

re.match('b', x) ✗

re.search('b', x) ✓

findall $\rightarrow$ returns list of groups

=) $abc @ example.com *$

$\wedge (1w+) @ (1w+) \wedge \cdot + \$$

findall returns $[(abc, example)]$

(tuple of groups)

05/08/2024

why we use brackets.

⇓

To force precedence

↓

readable → eliminate ambiguity.

eg:- ab^+

⇒ $ab, abb, abbb, \dots$

$(ab)^+$

⇒ $ab, abab, ababab, \dots$

<sentence id = '1'>

(CML - creative Markup language)

1 It I \t PRP

2 It am \t VAUX

3 It going. \t VM

4 \t home \t NN

5 \t . PUNC

⇓
This annotation format is called "SSF"

(Shanti standard format)

</sentences>

(CONLL) → line format

~~CONLL~~

Normal → I am going home.

CONLL
format

I It PRP

am It VAUX

going It VM

home It NN

. It PUNC

BLANK LINE

I PRP

study VM

at PREP

SUNIT NNP (Proper noun)

<Sentence id = 2>

1 I PRP

2 study VM

3 at PREP

4 Sunit NNP

5 . PUNC

<Sentence>

Regex

for header

(< sentence + id = '(id+)' >)

for text

(.*?)

for footer

(< / sentence >)

Parsing

→ Rule based system

↓
Extracting Part

↓
KEY word Extraction

↓
Patterns.

After tokenization

1) Standardization

or

Normalization.

↓

Case Normalization (lower case or upper case)

Substitution

Sub (pattern , replacement , string)

x = "I am going."

Pattern → need to search in string

sub('am', 'was', x)

x = "we ~~were~~ are going."

sub('am|are', 'were', x)

sub('are', 'were', x)

for both we use → '(am|are)'

but for replacement we can't write like above, so need a function for that

⇒ def repl(match-object):

if match-object.group(1) == 'am':

return 'was'

elif match-object.group(1) == 'are':

return 'were'

else: return None

List of unhealthy words = [- , - , - , - , -]

ELIZA → Joseph Weizenbaum, MIT Lab
I am '(.*?)'

User 1: I am depressed

ELIZA1: why are you depressed?

User 2: My father always scolds me.

ELIZA2: Give me an instance.

User 3: hmm

ELIZA3: I'm sorry, I can't understand your query.

works on

Infinite state machine / automata (on/a)
(finite state automata)

→ recognizer (recognise a string). (Acceptor)

fsa
├── Deterministic
└── Non-deterministic

sheep talk

baa!

baad!

baaaa!

baaaaa!

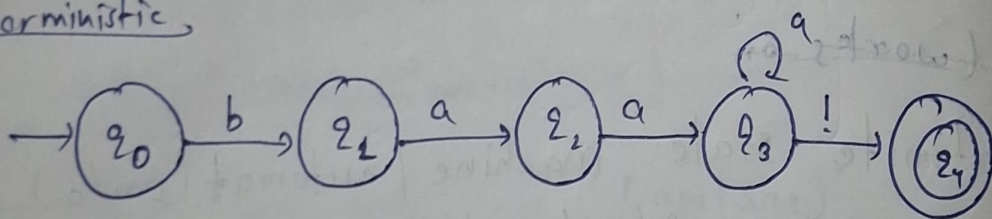
=> baa+!

$$M = (Q, \Sigma, \delta, q_0, f)$$

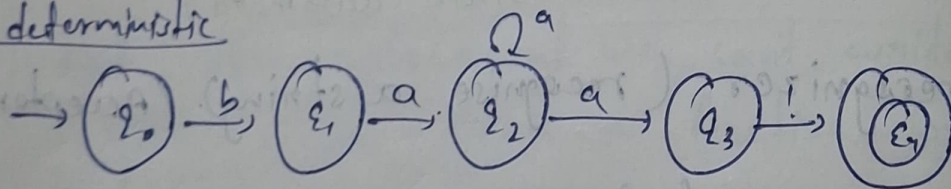
$\uparrow$ $\uparrow$ $\uparrow$
 state input start
 alphabet state

Transition fn set of final sets

Deterministic

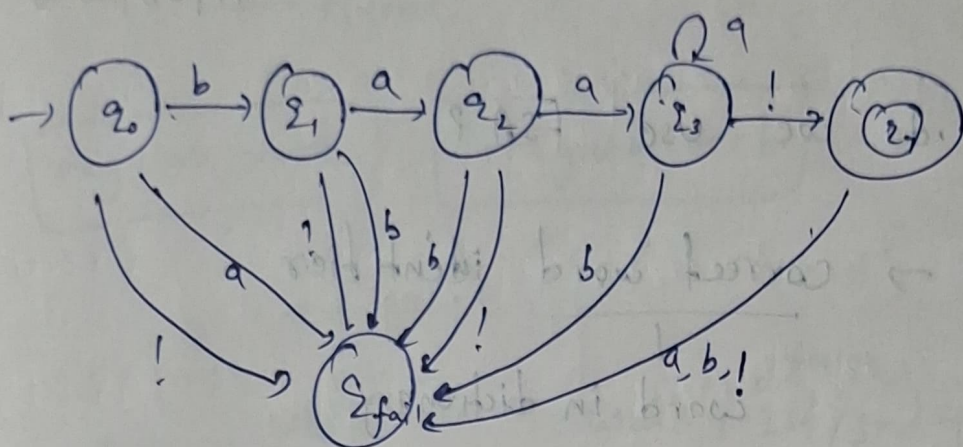


Non deterministic



	a	b	!
q_0	ϕ	q_1	ϕ
q_1	q_2	ϕ	ϕ
q_2	q_3	ϕ	ϕ
q_3	q_3	ϕ	q_4
q_4	ϕ	ϕ	ϕ

for better representation



function D-recognize (input, transition-table, initial-state, final-states)
returns accept or reject

current-state $\leftarrow$ initial-state

index $\leftarrow 0$

while true do

if current

if index == len(input-1) and current-state in final-states then

return accept

else if

~~current-state~~ transition-table[current-state, input(index)]
is empty

return reject

else:

current-state $\leftarrow$ transition-table[current-state, input(index)]

index $\leftarrow$ index + 1

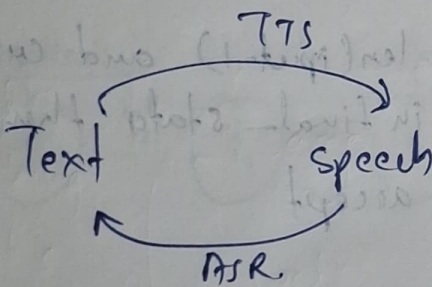
11/08/2026

Why do we use FSA?

FSA $\rightarrow$ correct word identifier
word in dictionary

Parsing:- Is the process of extracting/Providing grammatical/linguistic ~~information~~ ^{information} of a word

Morphological Analysis (morph analysis) } reverse of
morph generation } each other



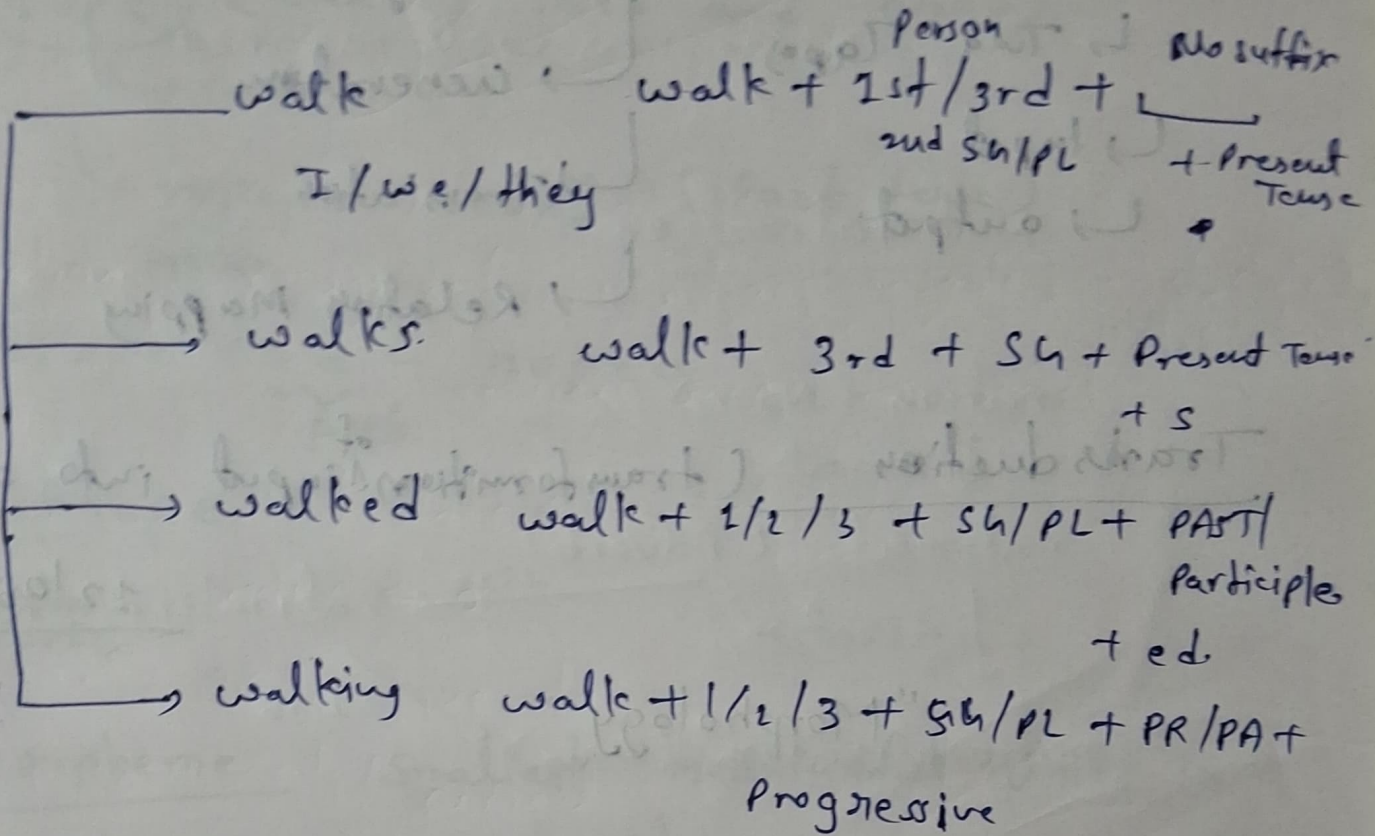
Morphological Analysis (cats = cat + N + PL)

morph generation (cat + N + PL = cats)

Noun { Singular
Plural.

regular verb

irregular verb



Case

kartā/nominatā

karmā,

karan

sampradān

Apadān

Sambandh

Adhikāran

Sambodhan

मैं

वही

से

के लिए

से (अलग होना)

वा/की/के

मे/पर

है/और

प्रतिबन्ध
अबोध
बोध

FSA (only one tape)

↳ just a recognizer

FST (Transducer) → Recognition

↳ Two Tapes

↳ Input

↳ Output

↳ Generation

↳ Translation

↳ Relation Mapping

Transduction (Transformation of input into output)

12/08/2020

"Morphology"

walk
walking
walks
walked

↳ Same root

walk

LEMMAIZATION

(STEM)MING

rope (can be verb and Noun).
root → Noun

roped (verb) (Past tense)
~~rope~~

rope + ed = roped
~~rope~~

unrope

~~suffix~~
prefix

Morpheme

Smallest meaningful units in a word

cats = Cat + -s

↓
stem/root

↓
suffix (inflection)
inflection (al)?

unhappy = un- + happy

↓
Prefix

(derivation)

derivation (al)?

10

In inflectional morphology the inflections do not change category of the word.
(meaning doesn't change)

Derivational morphology the category may change as well as meaning.

Nationalization

nation (Noun)

↓ +al

national (Adverb)

↓ +ization

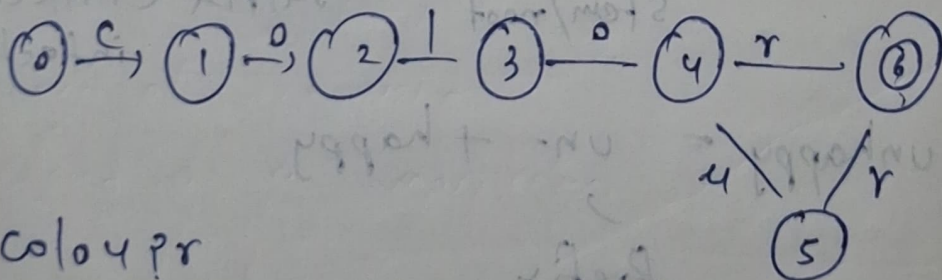
nationalization (Noun)

+ize → nationalize (Noun)

+tion

If a given word is valid

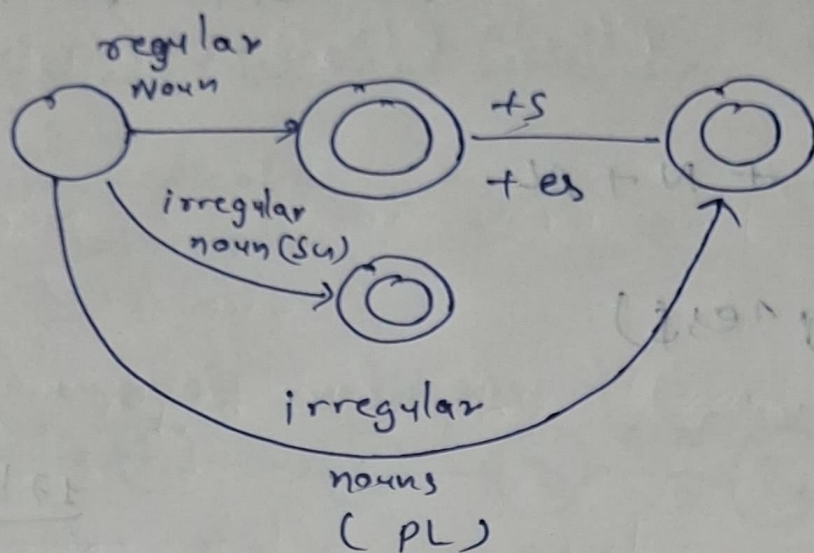
color



colorpr

colo (r/ur)

FSA for plural word recognition



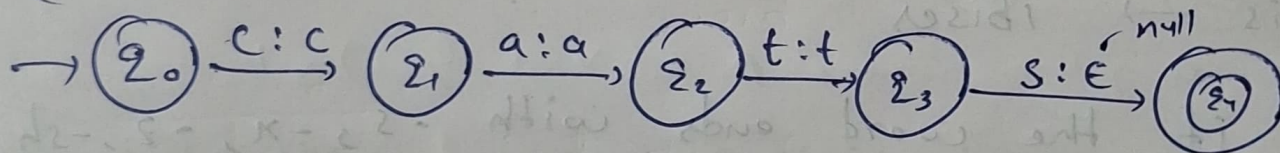
for FST

$\Delta \rightarrow$ output alphabet

$\delta \rightarrow Q \times \Sigma \rightarrow Q$

$\sigma : Q \times \Sigma \rightarrow \Delta$

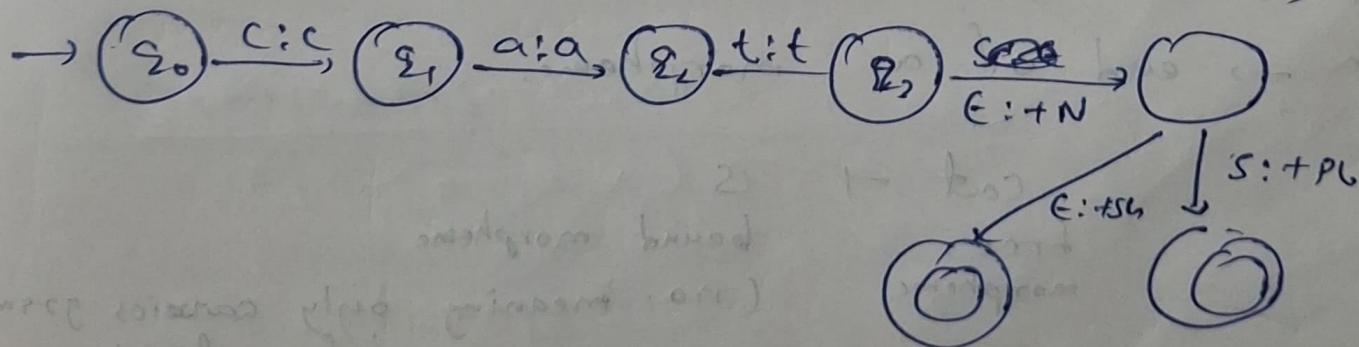
(cats)



$\text{cats} \rightarrow \text{cat} + N + \text{PL}$

(surface form)

$\text{cat} \rightarrow \text{cat} + N + \text{SG}$ (Lexical form)

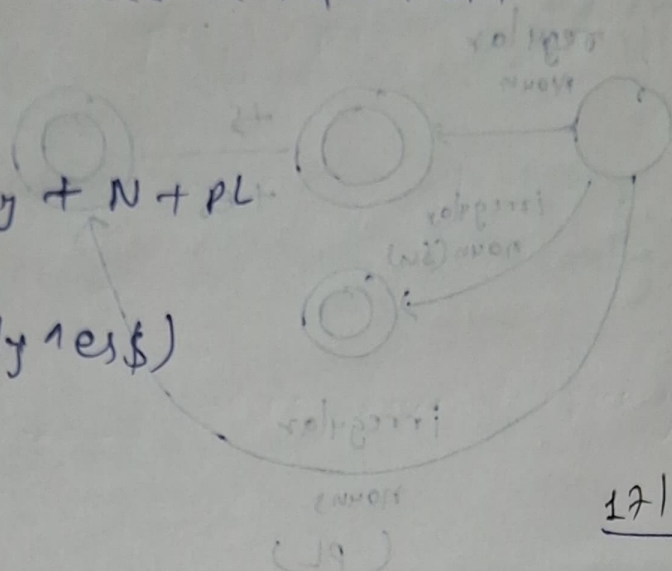


fly → flies

for this we need a intermediate form/tape

flies → fly + N + PL

↳ (flyes\$)



17/08/2020

T27

fox → foxes

box → boxes

crash → crashes

waltz → waltzes

ibis → ibises

rule: if the word ends with -s, -x, -z, -sh

(E-insertion) then add e before the plural marker

→ end of the word

^ → end of a morpheme

cat →

s

free
morpheme

bound morpheme

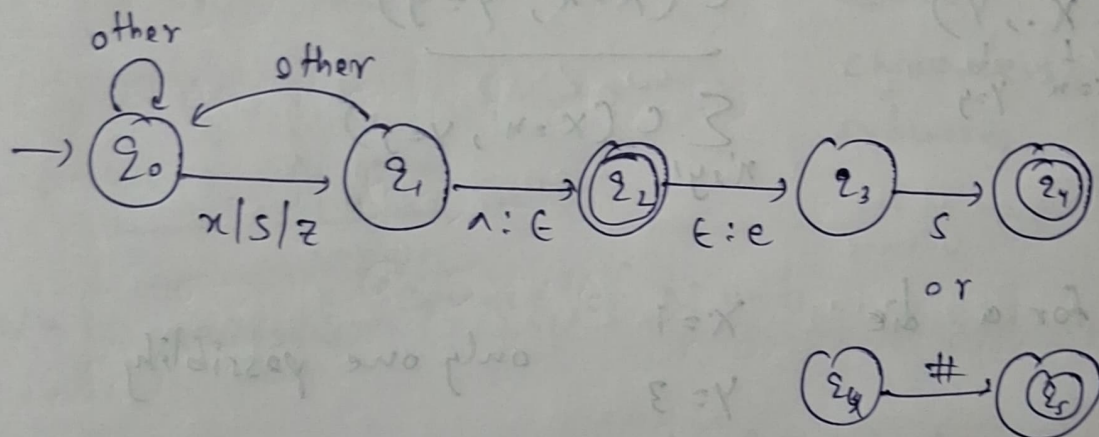
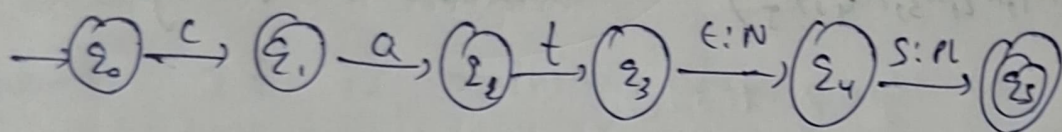
(no. meaning only carries grammatical information)

Surface $\leftarrow$ cats

intermediate $\leftarrow$ Catⁿ S #

lexical $\leftarrow$ cat + N + PL

↓
Grammatical
information



(Statistical)? Language model (ing)?

Next word prediction model.

* mathematics

$$X_1 \rightarrow X_2 \rightarrow X_3$$

$$P(x_1, x_2, x_3) = P(x_1) \times P(x_2 | x_1) \times P(x_3 | x_1, x_2)$$

Joint probability

↳ chain rule

$$P(x_6 | x_1, x_2, x_3, x_4, x_5) \rightarrow \text{Relative frequency.}$$

$$P(x_{i+1} | x_1, \dots, x_i) = \frac{c(x_{i+1} = x, y = y)}{\sum_{x', y'} c(x_{i+1} = x', y = y')}$$

for a die

$$X = 1$$

only one possibility

$$Y = 3$$

$$c(X=1, Y=1) = 1$$

$$\text{total} = 36$$

$$P(X=1, Y=3) = \frac{1}{36}$$

$$P(x_6 | x_1, x_2, x_3, x_4, x_5) = \frac{c(x_1, x_2, x_3, x_4, x_5, x_6)}{c(x_1, x_2, x_3, x_4, x_5)}$$

sparsity.

$$c(x=x) = \sum_{y' \in Y} c(x, y') \quad (\text{marginalization})$$

$$p(x=1) = p(x=1, y=1) + p(x=1, y=2) + \dots + p(x=1, y=6)$$

$$= \frac{1}{36} + \frac{1}{36} + \dots + \frac{1}{36} = \frac{6}{36} = \frac{1}{6}$$

encode

number $\leftarrow$ It is raining today.

N-gram

word

word N-gram

characters

subwords

N=1, unigram, Sequence of N units

N=2, bigram

$\text{len}(\text{sentence}) - N + 1$

N=3, trigram

N=4, quadgram

character N-gram

N=1 (with white space without white space)

for hindi $\rightarrow$ $\{w + \bullet\}$

eg: मानव only captures (म)

a) T(मानव) is not alphanumeric character,

we have to include $\{W + \bullet\}$

Statistical LM

19/08/2016

N-gram model

$\langle S \rangle w_1 w_2 \dots w_n \langle S \rangle \leftarrow$ bigram.
|
bos | eos
begin of the | end of the
sentence | sentence

$\langle S \rangle \langle S \rangle w_1 w_2 \dots w_n \langle S \rangle \langle S \rangle \leftarrow$ trigram

for a n-gram model, i want $(n-1)$ bos
and $(n-1)$ eos.

$$P(w_i | w_{i-1})$$

$$P_i(w_i | \langle S \rangle \langle S \rangle) \rightarrow P_i(w_i | w_{i-2} w_{i-1})$$

~~max~~ if the all probabilities are
given then

Select the word with maximum probability

$$\max(P(\omega_{start} | cs) cs) \rightarrow \text{The}$$

$$\downarrow$$

$$\max(P(\omega_2 | cs) the)$$

$\downarrow$

car

$$\max P(\omega_3 | the \text{ car})$$

$\vdots$

for N gram

$$\max P(\omega_i | \omega_{i-n+1} \omega_{i-1})$$

we calculate probability using "Relative frequency"

$$P(\omega_i | \omega_{i-2} \omega_{i-1}) = \frac{c(\omega_{i-2} \omega_{i-1} \omega_i)}{c(\omega_{i-2} \omega_{i-1})}$$

$$= \frac{P(\omega_{i-2} \omega_{i-1} \omega_i)}{P(\omega_{i-2} \omega_{i-1})}$$

$$= \frac{c(\omega_{i-2} \omega_{i-1} \omega_i)}{\sum_{\omega'} c(\omega_{i-2} \omega_{i-1} \omega')}$$

$$\sum_{\omega'} c(\omega_{i-2} \omega_{i-1} \omega')$$

S: The book is on the table.

$$c(\text{on the table}) = 0$$

$$p(w_i | w_{i-N+1}) = \frac{c(w_{i-N+1} \dots w_{i-1} w_i)}{c(w_{i-N+1} \dots w_{i-1})}$$

Smoothing (stop to be zero). (to avoid zero probability)

for that we do:

$$= \frac{c(w_{i-N+1} \dots w_{i-1} w_i) + 1}{c(w_{i-N+1} \dots w_{i-1}) + \underset{\downarrow V}{\text{constant}}}$$

V is the vocabulary of your language corpus

$$= \frac{c(w_{i-1} w_i) + 1}{c(w_{i-1}) + V}$$

$$c(w_{i-1}) = \sum_{w' \in V} (c(w_{i-1} w') + 1)$$

$$\sum_{w' \in V} c(w_{i-1}, w') + 1$$

$$= \sum_{w' \in \text{Bigram}} c(w_{i-1}, w') + \sum_{w' \in \text{no Bigram}} c(w_{i-1}, w') + V \times 2$$

$\xrightarrow{c(w_{i-1}) + V}$

24/08/2026

$$\Rightarrow p(w_i | w_{i-2}) = \frac{c(w_{i-2}, w_{i-1}, w_i) + 1}{c(w_{i-2}, w_{i-1}) + V}$$

} add 1 smoothing
Laplace smoothing

$\swarrow \quad \searrow$
 Set 1 Set 2

$c(w_{i-2}, w_{i-1}, w_i) > 0$ $c(w_{i-2}, w_{i-1}, w_i) = 0$

$$\sum_{w' \in V} c(w_{i-2}, w_{i-1}, w') + 1$$

$$= \sum_{w' \in V} c(w_{i-2}, w_{i-1}, w') + \sum_{w'' \in V} c(w_{i-2}, w_{i-1}, w'') + \sum_{w \in V} 1$$

$\searrow \quad \quad \quad \searrow \quad \quad \quad \searrow$
 $c(w_{i-2}, w_{i-1}) \quad \quad \quad 0 \quad \quad \quad V$

eg:-

$$c(w_{i-1}, w_i) = 30$$

$$c(w_{i-1}) = 100$$

$$p(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})} = \frac{30}{100} = 0.3$$

Add 2 smoothing

$$p(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i) + 1}{c(w_{i-1}) + 1000000}$$

$$= \frac{30 + 1}{1000000 + 1} = 0.0000309969$$

$$= 0.000031$$

$$= 3 \times 10^{-5}$$

Discounting (d)

$$c'(w_{i-1}, w_i) = \left(\frac{c(w_{i-1}, w_i) + 1}{c(w_{i-1}) + 1000000} \right) \times c(w_{i-1}, w_i)$$

Add-k-smoothing

$$p(w_i | w_{i-1}) = \frac{c(w_{i-1}w_i) + k}{c(w_{i-1}) + kv}$$

$$0 \leq k \leq 1$$

$$k \in [0.02, 0.5]$$

⇒ How to find the best value of k ?

GRID search

(not an exhaustive search)
→ having limited grid size

$$k_i = k_{i-1} + 0.05$$

Parameter (k)

take n values of k and find out the performance matrix.

⇒ $p(w_i | w_{i-1})$ → find it without smoothing

⇒ $p(w_i | w_{i-1})$ → find ^{it with} the smoothing using Add k

⊛ find the absolute difference (probability difference)

or

discounting it

$$|c'(w_{i-1}w_i) - c(w_{i-1}w_i)|$$

→ count

difference

★ we have to reduce $(kv) \in [20000 - 500000]$
 (very large)

we use

$$p(w_i | w_{i-2} w_{i-1}) = \frac{c(w_{i-2} w_{i-1} w_i) + 1}{c(w_{i-2} w_{i-1}) + N(w_{i-2} w_{i-1})}$$

Witten - Bell Smoothing

How to find the best value of k to calculate $p(w_i | w_{i-2} w_{i-1})$

$$p(w_i | w_{i-2} w_{i-1}) = \frac{c(w_{i-2} w_{i-1} w_i) + 1}{c(w_{i-2} w_{i-1}) + N(w_{i-2} w_{i-1})}$$

using limited prior

$$N(w_{i-2} w_{i-1}) = |\{w' \in V \mid \text{count}(w_{i-2} w_{i-1} w') > 0\}|$$

Types of

types of trigram which has history $(w_{i-2} w_{i-1})$

find it without smoothing $p(w_i | w_{i-1})$

find the smoothing with add $p(w_i | w_{i-2} w_{i-1})$

find the absolute difference (probability difference)

to calculate it $|p(w_i | w_{i-1}) - p(w_i | w_{i-2} w_{i-1})|$

25/08/2024

He had never seen a man die in his life

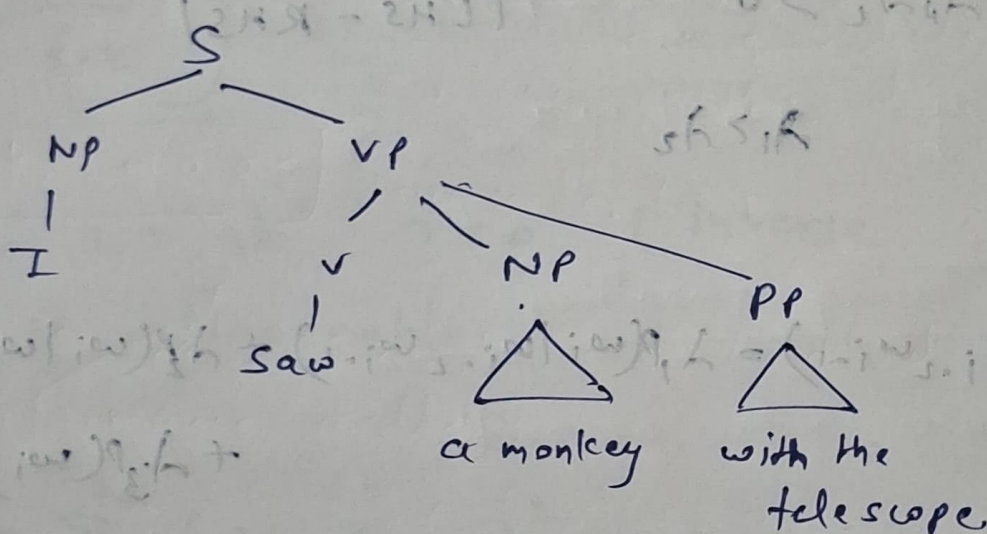
I saw a monkey with the telescope



PP

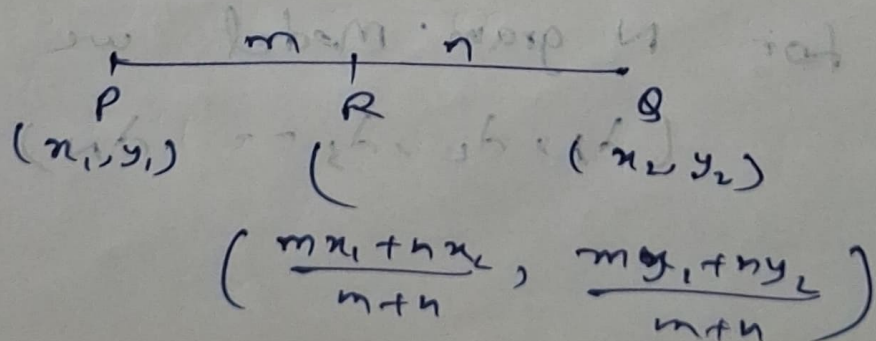
attachent issue

Prepositional
Phrase



MLE $\rightarrow$ maximum likelihood distribution.

Interpolation



let $\lambda = \frac{m}{n}$ $0 < \lambda < 1$

$$P(w_i | w_{i-1}) = \lambda_1 P(w_i | w_{i-1}) + \lambda_2 P(w_i)$$

LHS given $\lambda_1 + \lambda_2 = 1$ RHS

Grid Search (on development set)
 hold out set

$$\lambda_1, \lambda_2 > 0$$

$$|LHS - RHS|$$

$$\lambda_1 > \lambda_2$$

$$P(w_i | w_{i-2} w_{i-1}) = \lambda_1 P(w_i | w_{i-2} w_{i-1}) + \lambda_2 P(w_i | w_{i-1}) + \lambda_3 P(w_i)$$

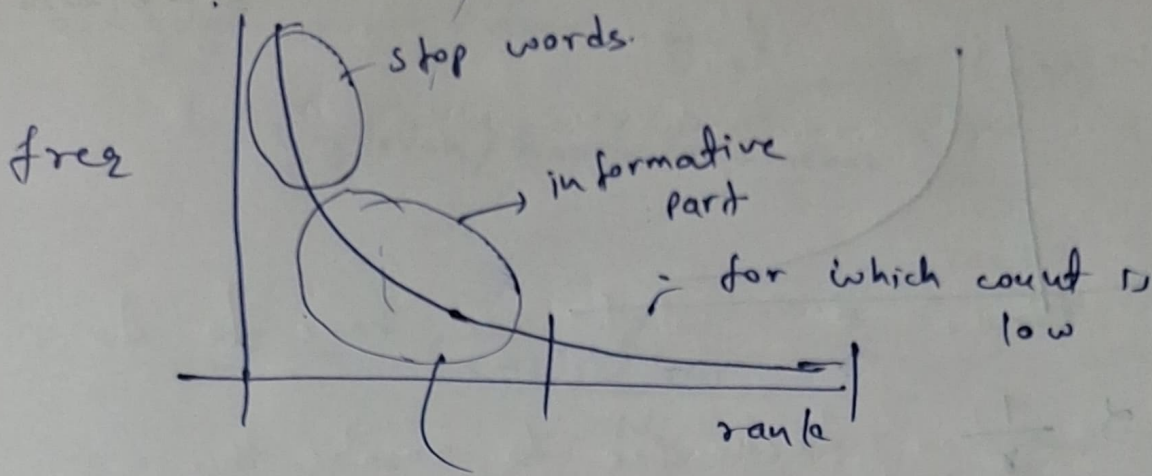
$$\lambda_1 + \lambda_2 + \lambda_3 = 1 \quad - \lambda_1, \lambda_2, \lambda_3 > 0$$

$$\lambda_1 > \lambda_2 > \lambda_3$$

$\Rightarrow$ for N gram model we need N λ 's

$$(\lambda_1, \lambda_2, \lambda_3, \dots, \lambda_N)$$

$$\left(\frac{\lambda_1 + \lambda_2 + \dots + \lambda_N}{N}, \frac{\lambda_1 + \lambda_2 + \dots + \lambda_N}{N} \right)$$



(Zipf's curve)

$$\text{rank} \propto \frac{1}{\text{freq}}$$

$$c = ab^r \quad \text{inverse law}$$

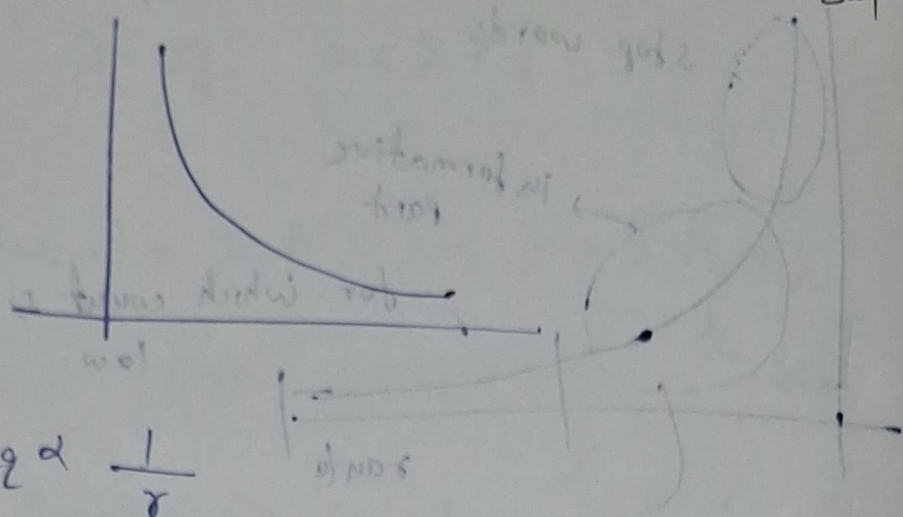
$$c = \frac{a}{b^r} \quad (\text{learnable})$$

$$\min \sum_{i=1}^N (c_i - c_{\text{pred}})^2$$

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$



freq $\propto \frac{1}{r}$

freq = $a b^{-r}$ (given this)

$\log(\text{freq}) = \log a - r \log b$

$y = w_0 + w_1 x$

fit > data

Good Turing smoothing

it takes into account

freq of freq (bin² - 1) $\sum_{i=1}^n n_i n_i$

x	1
y	1
z	1
a	2
b	2
c	3

x
y
z
1

$N_1 = 3$

a
b
2

$N_2 = 2$

c
3

$N_3 = 1$

u
v

 ← unseen

n-gram with frequency = 1

↳ singleton/hapax legomenon

MLE
of
count

→ C

$$C^* = (C+1) \frac{N_{C+1}}{N_C}$$

in
word

→ C*

Tuning

$$C_1^* = \frac{(1+1) N_2}{N_1}$$

$$= \frac{(2)(2)}{3} = \frac{4}{3}$$

$$C_2^* = \frac{(2+1) N_3}{N_2} = \frac{3 \times 1}{2} = \frac{3}{2}$$

$$\frac{C^*}{3} = \frac{(\cancel{3+1})}{3}$$

Total probability mass which is reserved by unseen.

$$\text{Total } P_r \text{ (unseen)} = \frac{N_1}{N}$$

↳ total no. of n-grams

$$= \frac{3}{10} = 0.3$$

$$P_{h7}^*(\text{unseen}) = \frac{\left(\frac{N_1}{N}\right)}{\text{total unseen}} = \frac{0.3}{2} = 0.15$$

$\Rightarrow$ when $N_{c+1} = 0$ what we have to do
we have two technique to resolve this

① Averaging

$$c^* = \frac{N_c}{\left(\frac{\text{next-higher} - \text{Previous-lower}}{2}\right)}$$

next-higher $\rightarrow$ may not be $c+1$

Previous-lower $\rightarrow$ may not be $c-1$

② Hack

$$c^* = (c+1) \frac{N_{\text{next-higher}}}{N_c}$$

Approximation

$$c^* = \frac{(c+1) S(c+1)}{S(c)}$$

$$N_c = e^a \cdot b^c$$

$$\log N_c = a + c \log b \leftarrow \text{this is linear in log scale}$$

$$\downarrow$$
$$S(c)$$

we will learn this a, b

$$P_{HT}^*(w_i | w_{i-1}) = \frac{c^*(w_{i-1}, w_i)}{c(w_{i-1})}$$

conditional probability

Joint probability

$$P(w_{i-1}, w_i) = \frac{c^*(w_{i-1}, w_i)}{\text{Total no of bigrams}}$$

() don't want this

Back off $\rightarrow$ is also a smoothing technique which is very much related to interpolation

$$P_{b_0}(w_i | w_{i-1}) = \begin{cases} P_{MLE}(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}, & c(w_{i-1}, w_i) > 0 \\ P_{MLE}(w_i), & \text{otherwise} \end{cases}$$

$$P_{b_0}(w_i | w_{i-2} w_{i-1}) = \begin{cases} P_{MLE}(w_i | w_{i-2} w_{i-1}) & \text{if } \text{count}(w_{i-2} w_{i-1}, w_i) > 0 \\ P_{MLE}(w_i | w_{i-1}) & \text{if } \text{count}(w_{i-1}, w_i) > 0 \\ P_{MLE}(w_i), & \text{otherwise} \end{cases}$$

STUPID Back off

$$P_{b_0}(w_i | w_{i-1}) = \begin{cases} P_{MLE}(w_i | w_{i-1}) & \text{if } c(w_{i-1}, w_i) > 0 \\ 0.4 \times P_{MLE}(w_i), & \text{otherwise} \end{cases}$$

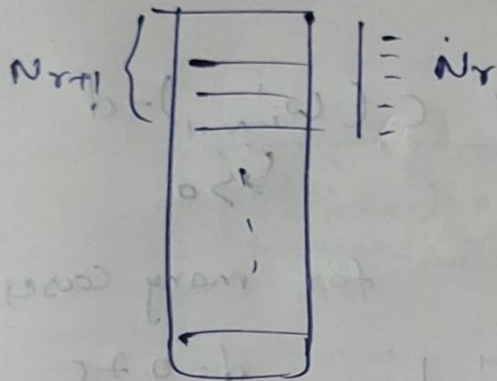
$$P_{MLE}(w_i) = \frac{\text{Count}(w_i)}{\text{Total words}}$$

for finding value of λ we use grid search.

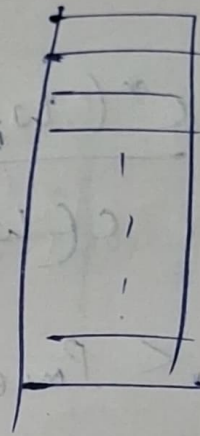
$$0 < \lambda < 1, \quad \lambda < 0.5$$

Good Turing

Training



Testing (leave one out)



If some of them go to ~~training~~ testing then it becomes
 $(N_{r+1} \rightarrow N_r)$

$$C^* = k \frac{N_{r+1}}{N_r}$$

KATZ Backoff $\rightarrow$ they used d from
Wood Turing $d = \frac{c^*}{c}$

Absolute discounting

$\Rightarrow$ Can I discount a fixed value

$\rightarrow$ from seen (higher) N-grams?

$$p^*(w_i | w_{i-N+1}^{i-1}) = \frac{c^*(w_{i-N+1}^i)}{c(w_{i-N+1}^{i-1})} \quad c(w_{i-N+1}^i) - d$$

$d > 0$

$$< p_{MLE}(w_i | w_{i-N+1}^{i-1})$$

for many cases

$$d = 0.75$$

for english corpus

Such as WST.

Wall Street Journal

Absolute Discounting

$\downarrow$
Kensler Neg smoothing.

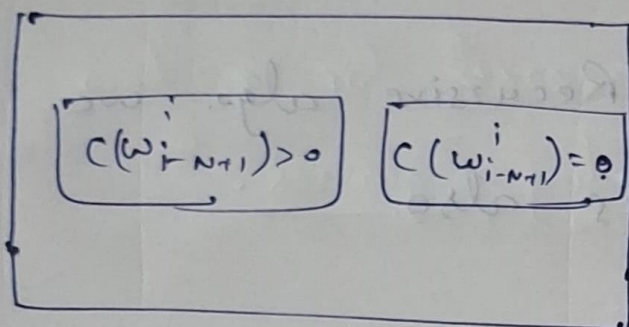
$$p_{KATZ}(w_i | w_{i-1}) = \begin{cases} p^*(w_i | w_{i-1}), & c(w_{i-1}, w_i) > 0 \\ k \times p^*(w_i), & \text{otherwise} \end{cases}$$

$k \rightarrow$ Normalizing factor

$d \rightarrow$ can be absolute or found from G.T.

$k \rightarrow$ represent what amount of mass is reserved for this lower n -gram

$$\text{total mass discounted} = 1 - \sum_{\substack{\text{for lower } n\text{-grams} \\ (i-1) \leq j < i}} p^*(w_i | w_{i-N+1}^{i-1})$$



$$P_{\text{KATZ}}(w_i | w_{i-1}) = \begin{cases} p^*(w_i | w_{i-1}), & c(w_{i-1}, w_i) > 0 \\ \alpha(w_{i-1}) \times p^*(w_i), & \text{otherwise} \end{cases}$$

$$P_{\text{KATZ}}(w_i | w_{i-N+1}^{i-1}) = \begin{cases} p^*(w_i | w_{i-N+1}^{i-1}), & \text{if } c(w_{i-N+1}^i) > 0 \\ \alpha(w_{i-N+1}^{i-1}) \times P_{\text{KATZ}}(w_i | w_{i-N+2}^{i-1}), & \text{otherwise} \end{cases}$$

$$d_{i-N+1}^{i-1} = \frac{1 - \sum_{\omega_{i-N+1}: c(\omega_{i-N+1}) > 0} p^*(\omega_i / \omega_{i-N+1}^{i-1})}{\sum_{c(\omega_{i-N+1}) = 0} p^*(\omega_i / \omega_{i-N+2}^{i-1})} \quad \left(\begin{array}{l} \text{Recursive} \\ \text{algorithm} \end{array} \right)$$

Normalizing factor

$$\sum_{c(\omega_{i-N+1}) = 0} p^*(\omega_i / \omega_{i-N+2}^{i-1})$$

d/c^* can be computed from h.T.

★ because it is a Recursive algo. we need base cases also.

$$p_{katz_2}(\omega_i / \omega_{i-2} \omega_{i-1}) = \begin{cases} p^*(\omega_i / \omega_{i-2} \omega_{i-1}) & \text{if } c(\omega_{i-2} \omega_{i-1} \omega_i) > 0 \\ \alpha(\omega_{i-2} \omega_{i-1}) \times p_{katz_2}(\omega_i / \omega_{i-1}) & \text{if } c(\omega_{i-1} \omega_i) > 0 \\ \cancel{p^*(\omega_i)} & \text{otherwise,} \end{cases}$$

$$p_{katz_2}(\omega_i / \omega_{i-1}) = \begin{cases} p^*(\omega_i / \omega_{i-1}) & \text{if } c(\omega_i / \omega_{i-1}) > 0 \\ \alpha(\omega_{i-1}) \times p^*(\omega_i) & \text{otherwise,} \end{cases}$$

Absolute discounting

$$p_{AD}(\omega_i / \omega_{i-N+1}^{i-1}) = \frac{\max(c(\omega_{i-N+1}^i) - d, 0)}{c(\omega_{i-N+1}^{i-1})} + \lambda(\omega_{i-N+1}^{i-1}) p_{AD}(\omega_i / \omega_{i-N+2}^{i-1})$$

Normalizing factor

$$\frac{d}{c(\omega_{i-N+1}^{i-1})} \times \left| \left\{ \omega : c(\omega_{i-N+1}^{i-1}, \omega) > 0 \right\} \right|$$

this is a set

This is how many n-grains got ~~we~~ discount

$$p_{AD}(\omega_i / \omega) = \frac{\max(c(\omega_{i-1}, \omega) - d, 0)}{c(\omega_{i-1}, \omega)} + \lambda(\omega_{i-1}, \omega) p_{AD}(\omega_i / \omega)$$

$$c(\omega_{i-1}, \omega) \left\{ \frac{c(\omega_{i-1}, \omega)}{c(\omega_{i-1}, \omega)} \sum_{\omega} \right\}$$

$$\left| \left\{ \omega : c(\omega_{i-1}, \omega) > d \right\} \right| \times b = c(\omega_{i-1}, \omega)$$

(cardinality of set)

$$\left[\frac{\left| \left\{ \omega : c(\omega_{i-1}, \omega) > d \right\} \right| \times b}{c(\omega_{i-1}, \omega)} \right]$$

KNESER NEY Smoothing

=> This is a better version of absolute Discounting

=> continuation probability

when I am reading a novel for a few hours,
I need my reading glasses.

Bigram Probability

$$P_{KN}(w_i | w_{i-1}) = \frac{\max(c(w_{i-1}, w_i) - d, 0)}{\sum_{w'} c(w_{i-1}, w')} + \lambda(w_{i-1}) * P_{KN}(w_i)$$

$$\lambda(w_{i-1}) = \frac{dx \left| \underbrace{\{w' : c(w_{i-1}, w') > 0\}}_{\text{Cardinality of set}} \right|}{\sum_{w'} c(w_{i-1}, w')}$$

$$\lambda(w_{i-1}) = \frac{dx \left| \{w' : c(w_{i-1}, w') > 0\} \right|}{\sum_{w'} c(w_{i-1}, w')}$$

$P_{KN}(w_i) \rightarrow P_{\text{continuation}}$

$P_{KN}(w_i) \rightarrow$ How many bigrams are continuing to end with w_i or continue till

$$P_{KN}(w_i) = \frac{|\{w' : c(w', w_i) > 0\}|}{|\{w, w' : c(w, w') > 0\}|}$$

How many unique bigrams

bigram type

$$P_{KN}(\text{francisco}) = \frac{|\{w : c(w, \text{francisco}) > 0\}|}{\text{total bigram types}}$$

for Trigram Probability

$$P_{KN}(w_i | w_{i-2} w_{i-1}) = \frac{\max(c(w_{i-2} w_{i-1} w_i) - d, 0)}{c(w_{i-2} w_{i-1}) + \lambda(w_{i-2} w_{i-1}) P_{KN}(w_i | w_{i-1})}$$

Higher model term

lower model term

$$P_{KN}(w_i | w_{i-N+1}^{i-1}) = \frac{\max(c(w_{i-N+1}^{i-1} w_i) - d, 0)}{c(w_{i-N+1}^{i-1})}$$

$$+ \lambda (\omega_{i-N+1}^{i-1}) P_{KN}(\omega_i | \omega_{i-N+1}^{i-1})$$

for unigram

$$P_{KN}(\omega_i) = \frac{|\{\omega' : c(\omega', \omega_i) > 0\}|}{\text{total bigram types}}$$

Good Turing we want reliable estimates for low ω counts.

$$c^* = \begin{cases} (c+1) \frac{N_{c+1}}{N_c} & \text{when } c \leq 5 \\ c & , \text{ otherwise} \end{cases}$$

$$(P_{GT})^* = \frac{\left(\frac{N_1}{N}\right)}{\text{total unseen n-grams.}}$$

$$c_i = \frac{c^*}{c_{\text{history}}}$$

Byte pair encoding (BPE)

↳ actually proposed as a compression technique

{ word piece
Sentence Piece } variants of BPE and more reliable.

(subwords)

Tokenized output for small dataset
(freq)

huggingface	5
hugs (hugging + face)	6
bugs	7
bun	4
bag	3
hugging	2

unique characters

$V_{init} = \{h, u, g, i, n, f, a, c, e, s, b\}$

h u g g i n g f a c e s
h u g s

b u g s

b u n

b a g

h u g g i n g

$$\text{count}(hu) = 13$$

$$\text{count}(ug) = 20. \text{ (most frequent)}$$

combine the most frequent pair of character together. before repeating the same process add the $\{ug\}$ to vocabulary.

$$V_1 = V_{\text{init}} \cup \{ug\}$$

$\Rightarrow$ now most frequent (hug)

$$V_2 = V_1 \cup \{hug\}$$

when to stop $\rightarrow$ if your vocab does not change.